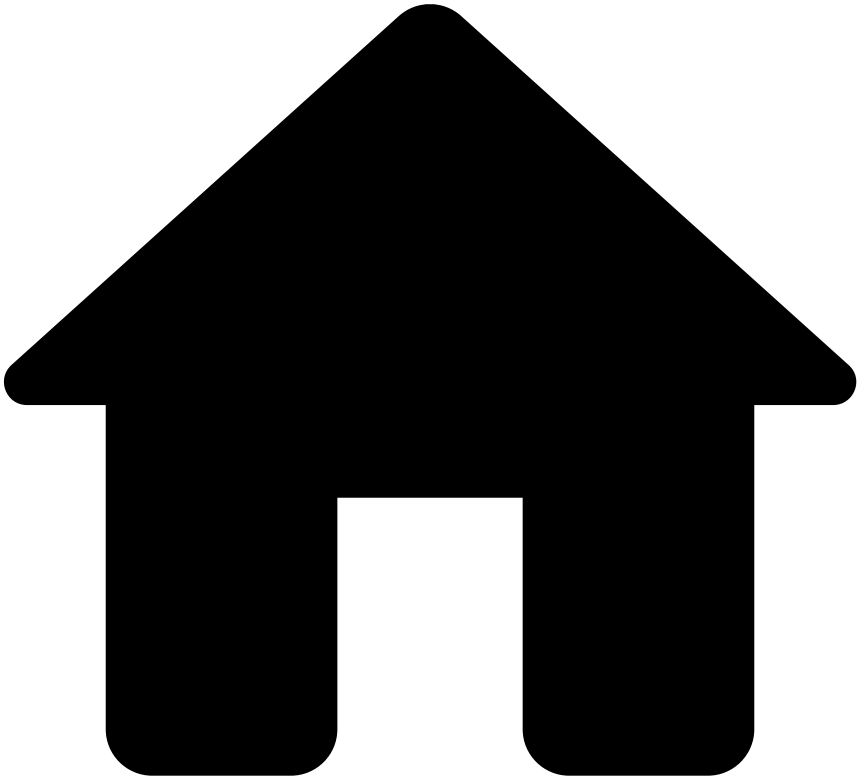


Configuration Overview

Setting up component encryption requires configuring encryption between internal Pulse components as well as between external components (e.g. between RabbitMQ brokers, or between Cassandra Nodes). On the Pulse side, this is done by setting a default encryption configuration in the pulse.global properties, which can be overridden for each component that require different configurations.

Configuring encryption between Pulse and an external component requires configurations both in Pulse and in the external component. The configuration of the properties in Pulse are generally the same, regardless of the external component whereas the configuration in external components is specific to the component. The table in the section Configuring external components summarizes the main configuration requirements for each external component. The page Configuring External Components describes the configuration requirements for external components in more detail.

The following sections give you an overview of the security concepts and settings for component encryption.

Understanding keystores and truststoresâ□□

Component encryption requires configuration of keystores on the server side and truststores on the client side. This applies regardless of the component; however, the format of the keystore and the truststore may vary between components. The following terminology is used to identify the different formats:

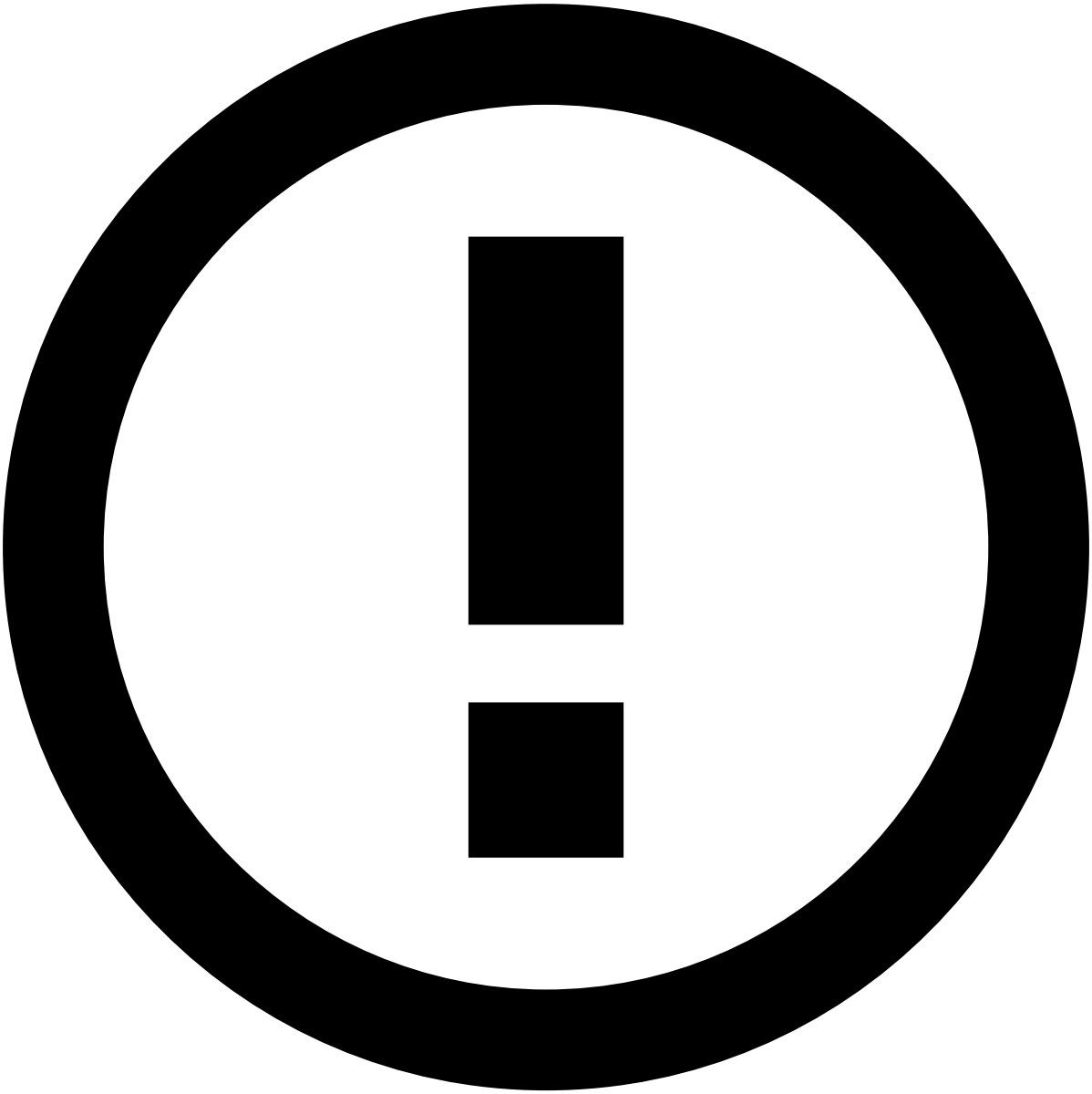
- **Java Truststore**: contains the public certificate of the CA that signs the server certificates in JKS format (.jks file). Is referred in this article as truststore.jks.
- **Java Keystore**: contains the server certificate signed by the CA and the server's private key (.jks file). Is referred in this article as keystore.jks.
- **Certificate Authority** (CA): contains the same as a truststore, but in a PEM format (.pem file). This format is required by Rabbit MQ. Is referred in this article as ca.pem.
- **Server certificate**: contains the same as a Keystore, but in PEM format (.pem file). Is referred in this article as certificate.pem.
- **Server certificate private key**: (.pem file). Referred in this article as key.pem.

The files mentioned above are used in the case when all certificates are signed by an internal CA and all components are configured to trust this CA (i.e. approach implemented in Pulse by default). Other scenarios are out of the scope of this page. In Pulse, the configuration files depend whether Pulse acts as a server or as a client. Pulse acts as a client in most cases in which Pulse server/application engine communicate with an external component (e.g. between Cassandra or RabbitMQ. It acts as a server in communications between Pulse internal components (e.g. between the application engine and the Pulse server with RMI).

To help the configuration process, Pulse includes scripts to help creating keystores and truststores. You can visit the page Creating Certificates for more information on how to configure these scripts.

Configuring Pulse propertiesâ□□

Component encryption is implemented in Pulse using a default configuration specifying the required files in the client and server side for all components. This default configuration can be overridden for any component. For example, if Cassandra needs a different file configuration, the properties can be overridden to create an exception for Casandra. With this approach, the Pulse properties of the components that share the same encryption configuration only need to be defined once.



Configuring custom properties

Some components require separate configuration not fully covered in the global encryption properties (e.g. relational databases). Visit the page Configuring External Components to learn more about this.

In the following example, all the components have default configuration for server authentication except for Cassandra, in which the encryption is disabled:

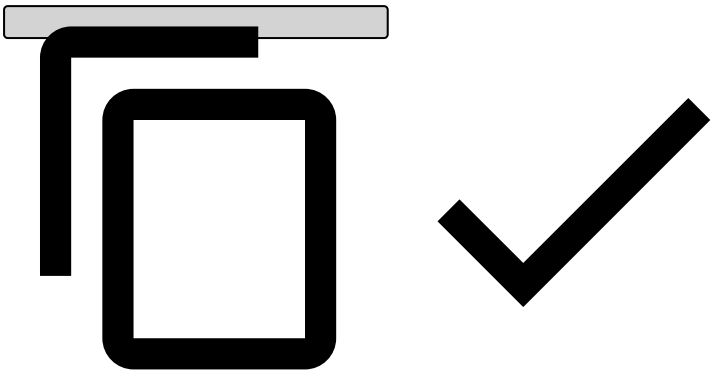
```
 # Default encryption properties
pulse.global.encryption.trustLevel=SERVER_AUTH
pulse.global.encryption.protocol=TLS

# Default encryption configuration for client-side components
pulse.global.encryption.client.truststore.file=cert/trusted-ca.jks
pulse.global.encryption.client.truststore.storepwd=trusted-ca-store-pwd

# Default encryption configuration for server-side components
pulse.global.encryption.server.keystore.file=cert/my-identity.jks
pulse.global.encryption.server.keystore.storepwd=my-identity-store-pwd
```

```
pulse.global.encryption.server.keystore.keypwd=my-identity-key-pwd

# No encryption on Cassandra
pulse.global.cassandra.encryption.trustLevel=ENCRYPTION_NONE
```



The configuration above specifies the passwords for the truststore on a client side (trusted-ca.jks) and the keystore on the server side (my-identity.jks). Similarly to database connection passwords, store passwords may be encrypted.

Authenticationâ□□

The authentication is specified in the trustLevel property as described in the above code block.

Pulse supports the following authentication:

- **Encryption Only:** Pulse does not validate the certificate presented by the server, and thus, truststores are not required on the Pulse side. A keystore is still required if RMI is to be encrypted since this component acts as a server. The value of the property is ENCRYPTION_ONLY.
- **Server Authentication:** Pulse validates the server certificate by checking if the certificate is valid and signed by a trusted entity. This requires a truststore on the client configuration and a keystore on the server configuration. The value of the property is SERVER_AUTH.
- **Client Authentication:** In addition to server authentication, clients present a certificate to the server and the server must to trust it. This requires an additional keystore on the client configuration and a truststore on the server configuration. The value of the property is CLIENT_AUTH.

Configuring external componentsâ□□

The following table provides a summary of the files that are required for each external component that communicates with Pulse.

The page Configuring External Components describes the configuration requirements for external components in more detail.

Origin (Client)	Destination (Server)	Client Configuration	Server Configuration
Pulse server, App Engine	Rabbit MQ	truststore.jks, truststore password	ca.pem, certificate.pem, key.pem
Rabbit MQ	Rabbit MQ	ca.pem	ca.pem, certificate.pem, key.pem
Pulse server, App Engine, Spark, any other java client app	Pulse server, App Engine	truststore.jks, truststore password	keystore.jks, server certificate key password, keystore password
Web Browser	Pulse server	should contain the certificate authority installed	keystore.jks, server certificate key password, keystore password
Pulse server, App Engine	Cassandra	truststore.jks, truststore password	keystore.jks, keystore password (needs to be the same as the server private key password)
Cassandra (inter-node communication)	Cassandra	truststore.jks, truststore password	keystore.jks, keystore password (needs to be the same as the server private key password)

Origin (Client)	Destination (Server)	Client Configuration	Server Configuration
Pulse server, App Engine	Spark	truststore.jks, truststore password	keystore.jks, keystore password
Pulse server, App Engine	Relational database	truststore.jks, truststore password	On Posgres: keystore.jks, keystore password. Other servers may have distinct requirements.

Next stepsâ□□

Now that you have basic knowledge about the default Pulse properties, check the page Configuring External Components for more information about configuring component encryption.